



# Institut Sains dan Teknologi Terpadu Surabaya

Jalan Ngagel Jaya Tengah 73 – 77, Surabaya 60284 , Indonesia

Telp. (031) 5027920 Fax. (031) 5041509

## PROPOSAL TUGAS AKHIR (TA)

Periode Bulan: \_\_ Januari \_\_ Tahun: \_\_ 2025 \_\_

Semester Gasal / Genap \*) Tahun ajaran \_\_ 2025 \_\_ / \_\_ 2026 \_\_

Program / Program Studi : ~~D3~~ / S1 \*) Informatika  
 Nama Mahasiswa : kelun kaka santoso  
 NRP Mahasiswa : 222117039  
 Bidang Keahlian (Major) : Network and Distributed System  
**Sekaligus menjadi major pilihan, dan kesalahan pengisian major mengakibatkan GAGAL Tugas Akhir dan Yudisium**  
 Judul Tugas Akhir :



**Aplikasi Multiplatform E-Commerce dan Manajemen Distribusi Barang Menggunakan Framework NestJS dan Sveltekit**

Jenis Tugas Akhir : ☐ Hardware ☐ Software ☐ Studi Literatur / Pengkajian / Analisa \*)  
 \*) Coret yang tidak perlu

Pembimbing Utama : Dr. Ir. F.X Ferdinandus, M.T.

Co. Pembimbing : \_\_\_\_\_

Jumlah SKS **SUDAH** LULUS : \_\_ 136 \_\_ SKS IPK : \_\_ 3.09 \_\_ ECC Level : \_\_ 4 \_\_

Mengetahui,

Surabaya, 28 Januari 2026

Pembimbing Utama,

Co. Pembimbing,

Pemohon,

( Dr. Ir. F.X Ferdinandus, M.T. )

( \_\_\_\_\_ )

( Kelun Kaka Santoso )

Catatan Tambahan:

---

---

---

---

---

---

---

---

Menyetujui,

Dekan,

Ketua Program Studi,

( Edwin Pramana, Ir., M.App.Sc., Ph.D. )

( Yosi Kristian, Dr., Ir., S.Kom., M.Kom. )

## **Hasil Keputusan Ketua Program Studi Periode FEBRUARI 2026**

### **Informasi Tugas Akhir/Tesis**

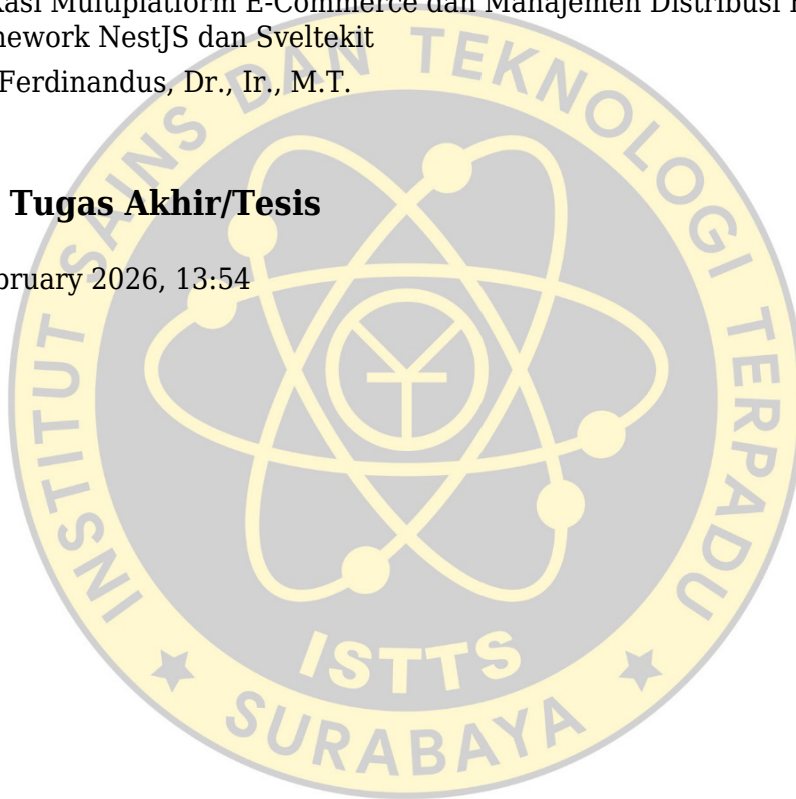
NRP : 222117039  
Nama : KELUN KAKA SANTOSO  
Judul : Pengembangan Sistem Informasi E-Commerce dan Manajemen Distribusi Barang  
Berbasis Multiplatform Menggunakan Framework NestJS dan SvelteKit  
Judul Baru : Aplikasi Multiplatform E-Commerce dan Manajemen Distribusi Barang menggunakan  
Framework NestJS dan Sveltekit  
Pembimbing : F.X. Ferdinandus, Dr., Ir., M.T.  
Co-Pembimbing :

### **Informasi Periode Tugas Akhir/Tesis**

Tanggal Cetak : 17 February 2026, 13:54

### **Hasil Revisi**

Judul Revisi :  
Kaprodi :  
Syarat :



## **Hasil Keputusan Sidang Proposal Periode FEBRUARI 2026**

### **Informasi Tugas Akhir/Tesis**

NRP : 222117039  
Nama : KELUN KAKA SANTOSO  
Judul : Pengembangan Sistem Informasi E-Commerce dan Manajemen Distribusi Barang Berbasis Multiplatform Menggunakan Framework NestJS dan SvelteKit  
Judul Baru : Aplikasi Multiplatform E-Commerce dan Manajemen Distribusi Barang menggunakan Framework NestJS dan Sveltekit  
Pembimbing : F.X. Ferdinandus, Dr., Ir., M.T.  
Co-Pembimbing :

### **Informasi Periode Tugas Akhir/Tesis**

Tanggal Sidang Proposal : 4 February 2026  
Tanggal Cetak : 17 February 2026, 13:54

### **Review**

Reviewer 1 : Yosi Kristian, Dr., Ir., S.Kom., M.Kom.  
Status : **DIPERBAIKI**  
Pesan :

1. Algoritma yang dipakai Terlalu Sederhana: Penggunaan Reorder Point (ROP) hanya rumus matematika sederhana, bukan algoritma kompleks.
2. Pada batasan sistem poin 4, Mhs menyebutkan penugasan kurir dilakukan secara manual oleh Admin. Ini seharusnya bisa ditangani dengan algoritma semester 4 (seperti Traveling Salesman Problem atau Auto-dispatch) yang bisa menjadi nilai jual utama mahasiswa Informatika.
3. Disebutkan penggunaan HTTP Request untuk fitur Live Driver Location, HTTP polling sebenarnya tidak efisien. Seharusnya bisa memakai WebSocket atau MQTT untuk "Sistem Terdistribusi" yang efisien.
4. Perbandingan dengan Shopee pada Tabel 1 agak kurang pas ("Apple to Orange"). Shopee itu marketplace (C2C/B2C), sedangkan sistem TA ini adalah Single-seller/Internal Fleet. Fokuskan perbandingan pada sistem manajemen logistik delivery sejenis (misal: Lalamove)
5. Karena major Anda ND, tambahkan pengujian Load/Stress Testing (misal menggunakan Apache JMeter atau k6) pada server NestJS Anda. Ukur latency saat menangani ratusan concurrent request pelacakan lokasi. Ini lebih berbobot daripada sekadar UAT.

Tanggapan :

1. Sudah ditambahkan penjelasan algoritma Nearest Neighbor pada halaman 6 dan 15.
2. Fitur sudah diganti menjadi Auto-dispatch yang dijelaskan pada halaman 13.





3. Protokol komunikasi sudah diganti menggunakan WebSocket pada halaman 15.
4. Aplikasi pembandingan sudah diganti menjadi Lalamove pada halaman 16.
5. Saran tidak diimplementasikan. Pengujian dibatasi pada UAT (25 User) karena sistem dirancang untuk pada operasional toko mandiri dengan trafik rendah (Low Traffic), bukan skala Marketplace. Penegasan batasan ada halaman 15.

Reviewer 2 : Edwin Pramana, Ir., M.App.Sc., Ph.D

Status : **DIPERBAIKI**

Pesan : 1. Saran Judul: Aplikasi Multiplatform E-Commerce dan Manajemen Distribusi Barang menggunakan Framework NestJS dan Sveltekit  
2. Cari Program Pembandingan yang lebih cocok untuk aplikasi E-Commerce and Manajemen Distribusi Barang. Shopee adalah platform E-Marketplace. Bedakan Aplikasi E-Commerce dan E-Marketplace.  
3. Jika aplikasi dibuat untuk Toko/UD/PT tertentu, wajib ada surat izin dari Toko/UD/PT tersebut. Jika tidak maka sebaiknya aplikasi dibuat open untuk toko/UD/PT manapun (boleh dibatasi oleh jenis usahanya).

Tanggapan :

1. Judul sudah disesuaikan dengan saran
2. Aplikasi pembandingan sudah diganti menjadi Lalamove (Logistik/Distribusi) pada halaman 16.
3. Lingkup sistem sudah dipertegas sebagai Open System (General Purpose) untuk toko ritel umum pada halaman 15.



Reviewer 3 : Mikhael Setiawan, S.Kom., M.Kom.

Status : **DIPERBAIKI**

Pesan : 1. Sebaiknya dicek dahulu apakah Google Maps API perlu berbayar atau tidak, kalau tidak salah kapan hari berbayar.  
2. Apakah distribusi yang dimaksud pada tugas akhir ini adalah assign kurir dan pengantaran barang ke customer saja? atau hingga distribusi barang antar gudang?  
3. Untuk notifikasi pertimbangkan menggunakan FCM  
4. Pertajam fitur tugas akhir anda, misalnya pada pengguna, bisa menampilkan history  
5. Apakah aplikasi ini memiliki target perusahaan yang akan menggunakan tugas akhir anda? atau bisa dipakai untuk toko-toko secara umum. Hati-hati karena beda produk yang dijual, beda juga manajemen stok nya, misalnya perlu multi satuan

Tanggapan :

1. Layanan peta sudah diganti menggunakan OpenStreetMap & OpenRouteService pada halaman 15.
2. Lingkup distribusi dipertegas hanya untuk pengiriman Toko ke Pelanggan pada halaman 15.
3. Sistem menggunakan WebSocket untuk notifikasi dan pelacakan real-time yang lebih responsif, dijelaskan pada halaman 15.
4. Sudah ditambahkan fitur Riwayat Transaksi & Status Pesanan pada halaman 14.
5. Sistem bersifat Open System dan sudah ditambahkan fitur manajemen Multi-satuan (Kg/Sak) pada halaman 13 dan 15.

Reviewer 4 : Suhatati Tjandra, Ir., M.Kom.

Status : **DIPERBAIKI**

Pesan

- : 1. "Menyediakan infrastruktur digital yang terpusat bagi pemilik toko mandiri untuk memantau seluruh rantai pasok", benarkah sistem ini melibatkan SCM? kalau ya, SCM nya dibahas dan diimplementasikan sampai mana? kenapa kok "manajemen distribusi" disebut secara terpisah? bukankah manajemen distribusi merupakan bagian dari SCM?
2. bahas "algoritma Reorder Point (ROP) di buku TA
3. yang dibahas di TA ini bukan "manajemen inventaris" tapi inventori
4. pastikan pembahasan ujicoba di buku TA sesuai dengan role user yang terlibat.
5. saran: berikan fasilitas pelacakan pengiriman yang dalam kota (real time tracking), dan pengelompokan daerah pengiriman

Tanggapan

:

1. Kalimat tujuan telah direvisi. Istilah "Rantai Pasok" diganti menjadi "pemenuhan pesanan" agar spesifik pada Halaman 2.
2. Penjelasan teori ROP sudah dibahas pada Halaman 3, dan implementasinya pada Halaman 13.
3. Seluruh istilah "Inventaris" telah diubah menjadi "Inventori" secara konsisten di seluruh dokumen
4. Tabel rencana uji coba sudah dirinci berdasarkan 3 aktor utama: Admin, Kurir, dan Pelanggan pada Halaman 17.
5. Sudah ditambahkan fitur Live Tracking dan Optimasi Rute pada Halaman 14 dan 16.

## Hasil Sidang Proposal

Status

: **DIPERBAIKI**



- Syarat :
1. Untuk pengiriman barang, harus bisa mengelompokkan daerah pengiriman. Gunakan algoritma TSP atau lainnya.
  2. Algoritma Reorder Point perlu dibahas di buku tugas akhir.
  3. Penggunaan HTTP Request untuk pelacakan lokasi dinilai tidak efisien; disarankan menggunakan teknologi seperti WebSocket, MQTT, atau FCM untuk notifikasi dan komunikasi real-time.
  4. Perbandingan dengan Shopee dianggap kurang tepat karena perbedaan model bisnis (marketplace vs. single-seller). Disarankan membandingkan dengan sistem logistik serupa seperti Lalamove. Tambahkan juga perbandingan dengan TA lain yang sejenis.
  5. Tambahkan multi-satuan minimal 2 level.
  6. Ujicoba customer minimal 25, kurir minimal 3.
  7. Jika aplikasi ditujukan untuk perusahaan tertentu, diperlukan surat izin; jika untuk umum, sistem harusnya bersifat open.

Tanggapan :

1. Sudah diterapkan algoritma Nearest Neighbor untuk pengelompokan rute otomatis, dijelaskan pada halaman 6 dan 16
2. Penjelasan teori ROP sudah dibahas lengkap pada halaman 6, dan implementasinya pada halaman 13.
3. Protokol komunikasi pelacakan lokasi sudah diganti menggunakan WebSocket pada halaman 16.
4. Aplikasi pembanding sudah diganti menjadi Lalamove agar sepadan pada halaman 16.
5. Sudah ditambahkan fitur konversi satuan bertingkat (Kg/Sak) pada halaman 13 dan 16.
6. Jumlah responden sudah disesuaikan menjadi 25 Pelanggan dan 3 Kurir pada tabel uji coba halaman 17.
7. Lingkup sistem sudah dipertegas sebagai Open System (General Purpose) untuk toko umum pada halaman 15.

- Saran :
- Disarankan untuk pengujian Load/Stress Testing (misal menggunakan Apache JMeter atau k6) pada server NestJS. Ukur latency saat menangani ratusan concurrent request pelacakan lokasi.
- Disarankan untuk menambahkan fitur seperti pelacakan pengiriman real-time dalam kota, pengelompokan daerah pengiriman, dan riwayat (history) bagi pengguna.

Tanggapan :

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....





## PROPOSAL TUGAS AKHIR

### **Aplikasi Multiplatform E-Commerce dan Manajemen Distribusi Barang Menggunakan Framework NestJS dan Sveltekit**

Nama : Kelun Kaka Santoso  
NRP : 222117039  
Jurusan / Prodi : Informatika  
Major : Network and Distributed System  
Dosen Pembimbing : Dr. Ir. F.X Ferdinandus, M.T.

#### **I. Latar Belakang**

Perkembangan teknologi di sektor ritel menuntut toko modern untuk beralih dari manajemen konvensional ke sistem digital yang terintegrasi demi meningkatkan efisiensi dan kepuasan pelanggan. Namun, terdapat kesenjangan teknologi yang signifikan antara marketplace besar dengan toko mandiri yang masih mengelola operasionalnya secara manual. Akibatnya, banyak toko mandiri kesulitan bersaing dalam hal kecepatan pelayanan, akurasi ketersediaan stok, dan kepastian pengiriman barang kepada konsumen.

Kendala operasional yang krusial meliputi pencatatan stok yang tidak real-time, sehingga memicu risiko kehabisan barang tanpa adanya peringatan dini bagi pemilik toko. Di sisi logistik, pengiriman menggunakan kurir internal seringkali tidak transparan karena minimnya bukti valid seperti lokasi GPS atau foto penerimaan barang. Selain itu, pengaturan jadwal pengiriman yang dilakukan secara manual rentan menyebabkan bentrokan slot waktu antar pelanggan dan ketidakakuratan estimasi waktu sampai.

Sebagai solusi, penelitian ini mengusulkan pengembangan sistem informasi E-Commerce dan manajemen inventori berbasis multiplatform. Sistem ini menyatukan proses penjualan, pemantauan stok, dan pengiriman dalam satu pintu. Keunggulan utamanya terletak pada integrasi data real-time antara aplikasi admin dan kurir, serta penerapan algoritma Reorder Point (ROP) untuk memberikan rekomendasi waktu pemesanan ulang barang secara akurat.

Selain masalah jadwal, kendala operasional lainnya adalah ketidakefisienan rute pengiriman. Saat ini, penentuan urutan pengiriman paket sepenuhnya bergantung pada intuisi kurir tanpa adanya sistem yang mengelompokkan area tujuan secara otomatis. Hal ini seringkali menyebabkan kurir menempuh jarak yang lebih jauh dari yang seharusnya (rute zigzag), yang berdampak pada pemborosan bahan bakar dan keterlambatan waktu sampai ke pelanggan. Oleh karena itu,

diperlukan implementasi algoritma optimasi rute untuk membantu kurir menentukan urutan pengiriman yang paling efisien

## **II. Tujuan**

Pemanfaatan framework modern NestJS dan SvelteKit dalam sistem ini tidak hanya memberikan arsitektur backend yang terstruktur dan performa antarmuka web yang responsif, tetapi juga menjamin keandalan sistem dalam menangani manajemen stok dan penjadwalan logistik yang kompleks. Adapun tujuan dari penelitian ini adalah sebagai berikut:

1. Mengembangkan sebuah sistem informasi E-Commerce dan manajemen distribusi barang terintegrasi berbasis Multiplatform (Web Admin & Aplikasi Mobile), yang mencakup tiga modul utama yaitu manajemen transaksi penjualan, pengelolaan inventori, dan layanan logistik pengiriman mandiri.
2. Mengimplementasikan mekanisme sinkronisasi data stok otomatis dan fitur manajemen slot waktu pengiriman (*delivery scheduling*), yang berfungsi mencegah pemilihan jam ganda oleh pelanggan serta memberikan informasi estimasi waktu tiba (*Estimated Time of Arrival*) yang transparan dan akurat.
3. Menyediakan infrastruktur digital yang terpusat bagi pemilik toko mandiri untuk memantau seluruh alur pemenuhan pesanan (order fulfillment), mulai dari verifikasi pembayaran otomatis, pengelolaan stok toko, hingga validasi status pengiriman, guna meningkatkan efisiensi operasional toko dan kepercayaan pelanggan.
4. Menerapkan mekanisme Auto-dispatch untuk penugasan kurir secara otomatis dan algoritma Nearest Neighbor untuk mengoptimalkan urutan rute pengiriman berdasarkan jarak terdekat guna meningkatkan efisiensi distribusi barang.

## **III. Teori Penunjang**

Beberapa teori yang menunjang pembuatan tugas akhir ini adalah sebagai berikut:

### **1. NestJS**

NestJS adalah sebuah framework untuk membangun aplikasi sisi server (*server-side*) Node.js yang efisien dan dapat diskalakan (*scalable*). Framework ini dibangun dengan dan mendukung penuh TypeScript, namun tetap memungkinkan penggunaan JavaScript murni. NestJS menggabungkan elemen-elemen dari Pemrograman Berorientasi Objek

(OOP), Pemrograman Fungsional (FP), dan Pemrograman Reaktif Fungsional (FRP).

Dalam arsitektur sistem ini, NestJS berfungsi sebagai Backend utama yang menangani seluruh logika bisnis, seperti perhitungan stok, manajemen pesanan, dan penjadwalan pengiriman. Struktur modular NestJS sangat mirip dengan Angular, yang memungkinkan pengembang untuk mengorganisir kode ke dalam modul-modul terpisah (misalnya: Modul Auth, Modul Produk, Modul Logistik), menjadikan kode lebih rapi, mudah diuji, dan mudah dipelihara dibandingkan framework Node.js konvensional.

## 2. SvelteKit

SvelteKit adalah *meta-framework* untuk membangun aplikasi web modern yang dibangun di atas *library Svelte*. Berbeda dengan *framework* tradisional seperti React atau Vue yang melakukan sebagian besar pekerjaan di *browser* menggunakan *Virtual DOM*, SvelteKit melakukan proses kompilasi pada tahap *build time*. Pendekatan ini mengubah komponen deklaratif menjadi kode JavaScript imperatif yang sangat efisien yang secara langsung memanipulasi DOM.

Dalam proyek ini, SvelteKit digunakan untuk membangun *Dashboard Admin Web*. Keunggulan utamanya adalah performa yang sangat ringan dan cepat, memungkinkan admin toko untuk memuat halaman laporan penjualan yang padat data dan memantau stok barang tanpa lag, memberikan pengalaman pengguna yang responsif.

## 3. Flutter & Dart

Flutter adalah *Software Development Kit (SDK)* antarmuka pengguna (UI) *open-source* yang dikembangkan oleh Google untuk membangun aplikasi yang dikompilasi secara *native* (asli) untuk seluler (Android), web, dan desktop dari satu basis kode tunggal. Flutter menggunakan bahasa pemrograman Dart, yang dioptimalkan untuk klien yang cepat dan produktif.

Konsep inti Flutter adalah *widget*, di mana setiap elemen tampilan adalah widget yang dapat dikustomisasi. Dalam penelitian ini, Flutter digunakan untuk membangun Aplikasi Sisi Pengguna (*Customer*) dan Aplikasi Kurir. Kemampuan multiplatform Flutter memastikan bahwa aplikasi dapat berjalan mulus di berbagai perangkat smartphonedengan performa tinggi (60fps), yang sangat krusial untuk fitur peta interaktif dan pemindaian QR Code yang lancar.

## 4. Algoritma Reorder Point (ROP)

*Reorder Point (ROP)* adalah sebuah metode manajemen inventori yang menentukan tingkat stok minimum di mana pemesanan ulang barang harus dilakukan. Tujuannya adalah untuk memastikan bahwa stok baru tiba tepat sebelum stok yang ada habis, sehingga mencegah



terjadinya kekosongan barang (*stockout*) yang dapat merugikan penjualan. Rumus dasar ROP yang diterapkan dalam sistem ini adalah:

$$\text{Stock ROP} = (\text{Lead Time} \times \text{Rata-rata Penjualan Harian}) + \text{Safety Stock}$$

Penerapan algoritma ini pada backend sistem memungkinkan aplikasi untuk memberikan notifikasi otomatis kepada admin ketika stok suatu barang menyentuh angka kritis, menjadikan manajemen stok lebih cerdas dan efisien.

#### 5. PostgreSQL & Prisma ORM

*PostgreSQL* adalah sistem manajemen basis data relasional objek (ORDBMS) yang kuat dan *open-source*, dikenal dengan keandalan, ketahanan data, dan performanya dalam menangani kueri yang kompleks. Dalam proyek ini, *PostgreSQL* berfungsi sebagai tempat penyimpanan utama untuk data pengguna, transaksi, dan logistik. Untuk berinteraksi dengan database ini, sistem menggunakan Prisma, sebuah ORM (*Object-Relational Mapper*) generasi baru. Prisma menjembatani kode *TypeScript* di *NestJS* dengan database *PostgreSQL*, memungkinkan pengembang melakukan operasi CRUD (*Create, Read, Update, Delete*) dengan sintaks yang aman (*type-safe*) dan intuitif tanpa perlu menulis kueri SQL mentah secara manual, serta mempermudah manajemen relasi antar tabel (misalnya relasi antara tabel Pesanan dan Pengiriman).

#### 6. Penyedia Layanan Peta

Sistem tidak menggunakan layanan berbayar Google Maps Platform. Sebagai gantinya, sistem memanfaatkan OpenStreetMap (OSM) sebagai peta dasar (*basemap*) untuk visualisasi lokasi, serta menggunakan API OpenRouteService (ORS) untuk fitur perhitungan rute (*routing*) dan matriks jarak (*Distance Matrix*) yang diperlukan dalam algoritma optimasi pengiriman.

#### 7. Payment Gateway (Midtrans)

*Payment Gateway* adalah layanan perantara yang mengotorisasi pemrosesan pembayaran kartu kredit atau pembayaran langsung bagi bisnis e-commerce. Dalam sistem ini, integrasi dilakukan dengan Midtrans melalui API. Teknologi ini memungkinkan aplikasi untuk menerima berbagai metode pembayaran digital (seperti QRIS, *Virtual Account*, dan *E-Wallet*) secara otomatis.

Sistem ini memanfaatkan fitur *Webhook (HTTP Notification)* dari penyedia layanan pembayaran. Ketika pelanggan berhasil melakukan pembayaran, Midtrans akan mengirimkan sinyal ke backend *NestJS* secara real-time untuk memperbarui status pesanan menjadi "Dibayar" tanpa memerlukan verifikasi manual dari admin toko.

#### **8. Digital Signature (Canvas API)**

Tanda tangan digital dalam konteks aplikasi ini merujuk pada mekanisme penangkapan input sentuhan (*touch input*) pengguna pada layar perangkat seluler yang dikonversi menjadi format citra digital. Fitur ini diimplementasikan menggunakan pustaka berbasis HTML5 Canvas pada Flutter.

Teknologi ini merupakan komponen kunci dari fitur *Proof of Delivery* (PoD). Saat barang diterima, pelanggan dapat menggoreskan tanda tangan langsung di layar ponsel kurir. Data goresan tersebut disimpan sebagai gambar dan diunggah ke server sebagai bukti otentik serah terima barang yang tidak dapat disangkal (non-repudiation).

#### **9. Arsitektur Client-Server**

Arsitektur Client-Server adalah model komputasi terdistribusi yang membagi tugas antara penyedia sumber daya (Server) dan peminta layanan (Client). Dalam model ini, komunikasi dilakukan melalui jaringan komputer di mana klien mengirimkan permintaan (request) dan server memproses serta mengirimkan balasan (response). Dalam penelitian ini, arsitektur Client-Server menjadi fondasi utama topologi sistem yang dibangun. Aplikasi Mobile(Flutter) dan Web Dashboard (SvelteKit) bertindak sebagai sisi Client yang berada di lokasi fisik berbeda-beda (terdistribusi). Sementara itu, Server (NestJS) bertindak sebagai pusat pemrosesan logika bisnis dan basis data yang melayani permintaan dari banyak klien secara bersamaan, memungkinkan sinkronisasi data logistik antara kurir di lapangan dan admin di toko secara real-time.

#### **10. Location-Based Service (LBS)**

Location-Based Service (LBS) adalah layanan informasi yang dapat diakses melalui perangkat seluler dengan memanfaatkan kemampuan jaringan untuk menentukan posisi geografis pengguna. LBS menggabungkan teknologi navigasi, sistem informasi geografis, dan telekomunikasi untuk memberikan layanan yang kontekstual terhadap lokasi. Teori ini menjadi landasan ilmiah bagi fitur pelacakan armada (fleet tracking) dalam aplikasi. Sistem tidak hanya sekadar menampilkan peta, tetapi melakukan pengolahan data spasial secara terdistribusi. Posisi kurir (Latitude/Longitude) dikirimkan secara periodik melalui jaringan data seluler ke server, yang kemudian didistribusikan kembali ke dashboard admin untuk memberikan visualisasi pergerakan armada secara akurat.

#### **11. User Acceptance Testing (UAT)**

User Acceptance Testing (UAT) atau pengujian penerimaan pengguna adalah tahap akhir dalam siklus pengembangan perangkat lunak di mana sistem diuji oleh target pengguna sebenarnya sebelum diluncurkan. Tujuan utamanya adalah memvalidasi apakah alur sistem

sudah sesuai dengan kebutuhan bisnis (business requirements) dan dapat digunakan dengan nyaman dalam skenario operasional nyata. Dalam penelitian ini, metode UAT digunakan untuk mengukur tingkat keberhasilan aplikasi dari sudut pandang pengguna akhir. Pengujian melibatkan Admin Toko, Kurir, dan Pelanggan yang menjalankan skenario nyata mulai dari pemesanan hingga penyelesaian pengiriman. Hasil pengujian dievaluasi untuk memastikan bahwa fitur-fitur kompleks seperti pelacakan lokasi dan tanda tangan digital dapat diterima dan dioperasikan dengan baik oleh pengguna.

#### 12. **Algoritma Nearest Neighbor**

Algoritma Nearest Neighbor untuk Optimasi Rute *Nearest Neighbor* adalah algoritma optimasi rute yang bekerja dengan prinsip *Greedy*. Algoritma ini bertujuan untuk menentukan urutan kunjungan yang paling efisien dengan cara memilih lokasi tujuan berikutnya yang memiliki jarak terdekat dari posisi saat ini. Metode ini sangat efektif diterapkan pada sistem logistik untuk meminimalkan total jarak tempuh kurir dalam satu siklus pengiriman, serta memiliki waktu komputasi yang cepat sehingga tidak membebani kinerja aplikasi.

#### 13. **Mekanisme Auto-dispatch**

*Auto-dispatch* adalah mekanisme otomatisasi penugasan tugas kepada kurir tanpa intervensi manual. Sistem menentukan kurir yang paling tepat berdasarkan perhitungan skor yang menggabungkan parameter jarak terdekat (menggunakan rumus *Haversine*) dan ketersediaan armada. Dengan mekanisme ini, pesanan dapat langsung dialokasikan kepada kurir yang berada di radius terdekat dari lokasi toko untuk mempercepat proses penjemputan barang.

Commented [KS1]: Hasil Sidang Nomor 1

#### 14. **Reorder Point (ROP)**

*Reorder Point (ROP)* adalah titik ambang batas jumlah persediaan yang memicu tindakan pemesanan ulang stok barang. Penerapan algoritma ini bertujuan untuk meminimalkan risiko kehabisan stok (*stockout*) saat menunggu barang baru datang (*lead time*). Dalam sistem ini, notifikasi akan muncul secara otomatis ketika jumlah stok fisik menyentuh angka ROP yang telah dihitung berdasarkan rata-rata penjualan harian dan estimasi waktu pengiriman dari pemasok.

Commented [KS2]: Hasil Sidang Nomor 2

#### 15. **WebSocket dan Komunikasi Real-time**

WebSocket adalah protokol komunikasi komputer yang menyediakan saluran komunikasi dua arah (*full-duplex*) melalui koneksi TCP tunggal. Dalam penelitian ini, protokol WebSocket digunakan secara tunggal untuk menangani dua kebutuhan utama: mengirim koordinat GPS kurir ke server (*Live Tracking*) dan mengirimkan peringatan pesanan baru (*In-App Notification*) kepada kurir secara instan.

Commented [KS3]: Hasil sidang Nomor 3

#### IV. Ruang Lingkup

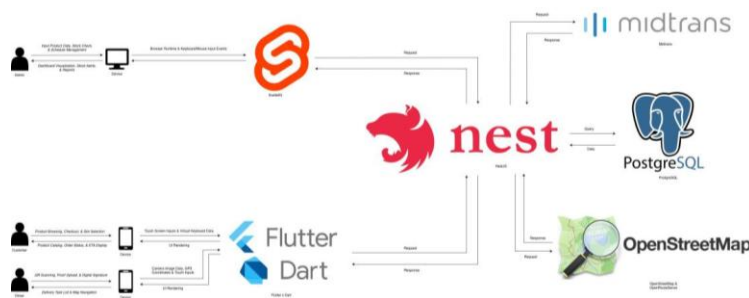
Pada bagian ruang lingkup ini akan dijelaskan beberapa penjelasan sistem yang meliputi arsitektur sistem, *use case* dari setiap aktor, aktivitas utama sistem, fitur program, dan batasan sistem. Ruang lingkup tersebut akan dijabarkan sebagai berikut:

##### A. Arsitektur Sistem

Sistem ini dirancang dengan menerapkan arsitektur *client-server* modern yang memisahkan tanggung jawab antara antarmuka pengguna (*frontend*) dan logika bisnis pusat (*backend*).

Seluruh logika bisnis utama, termasuk perhitungan algoritma *Reorder Point* (ROP), manajemen transaksi, dan pengaturan jadwal logistik, dipusatkan pada sisi server yang dibangun menggunakan framework NestJS. Server ini bertindak sebagai penyedia layanan (*API Provider*) yang aman dan terpusat.

Di sisi klien, sistem menyediakan dua *platform* antarmuka yang berbeda sesuai kebutuhan penggunaanya. Untuk Administrator Toko, disediakan aplikasi berbasis web yang dibangun dengan framework SvelteKit, yang menawarkan performa ringan dan cepat untuk pengelolaan data inventori yang padat. Sedangkan untuk Pelanggan dan Kurir, disediakan aplikasi *mobile* berbasis Flutter yang dikompilasi secara *native* (*Multiplatform*), memungkinkan akses fitur perangkat keras seperti kamera dan GPS secara optimal. Komunikasi data antara klien dan server difasilitasi melalui protokol HTTP menggunakan format pertukaran data JSON (*JavaScript Object Notation*) yang ringan.



**Gambar 1**  
**Arsitektur Sistem**

Pada Gambar 1, disajikan arsitektur sistem secara menyeluruh. Di sisi kiri, terdapat tiga aktor utama: Admin, Pelanggan, dan Kurir. Admin mengakses sistem melalui *Browser Desktop* yang menjalankan aplikasi



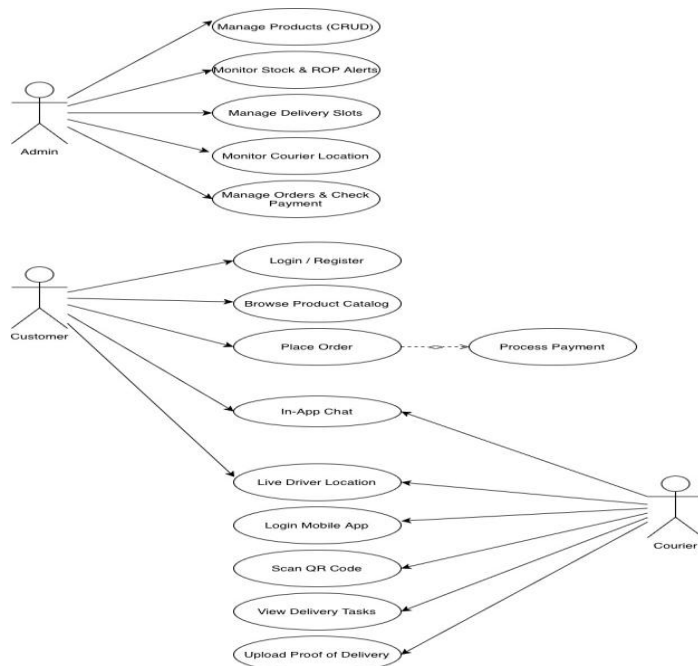
Web SvelteKit. Pelanggan dan Kurir mengakses sistem melalui *Smartphone* yang menjalankan aplikasi Mobile Flutter.

Kedua antarmuka klien (Web dan Mobile) tidak saling berkomunikasi secara langsung, melainkan terhubung ke satu titik pusat yaitu NestJS Server. Ketika Pelanggan melakukan pemesanan atau Kurir memperbarui status pengiriman, aplikasi klien mengirimkan *HTTP Request* (POST/GET/PUT) ke Server. Server kemudian memproses permintaan tersebut, melakukan validasi logika bisnis, dan berinteraksi dengan Database PostgreSQL untuk menyimpan atau mengambil data.

Selain itu, Server NestJS juga bertindak sebagai jembatan (*gateway*) ke layanan eksternal. Untuk pembayaran, server terhubung dengan API Midtrans untuk mendapatkan status transaksi. Untuk keperluan logistik, server memanfaatkan layanan API OpenRouteService untuk memvalidasi koordinat lokasi dan menghitung estimasi jarak tempuh. Hasil pemrosesan dari server kemudian dikirimkan kembali ke klien dalam bentuk HTTP Response berisi data JSON untuk ditampilkan kepada pengguna.

#### **B. Use Case Diagram**

Sistem ini memiliki tiga aktor utama yang berinteraksi dengan sistem, yaitu Admin Toko, Pelanggan, dan Kurir. Diagram Use Case berikut menggambarkan fungsionalitas tingkat tinggi yang dapat diakses oleh masing-masing aktor.



**Gambar 2**  
**Use Case Diagram**

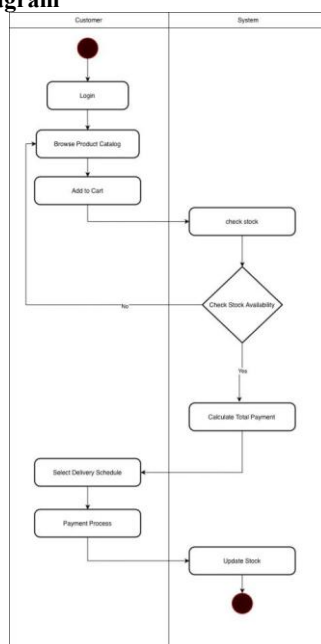
Berdasarkan Gambar 2 di atas, aktor Admin berperan sebagai pengelola utama sistem yang memiliki akses penuh ke *dashboard web*. Tugas utamanya meliputi manajemen data master produk, memantau ketersediaan stok yang terintegrasi dengan notifikasi *Reorder Point* (ROP), serta mengatur kuota slot jadwal pengiriman. Selain itu, Admin bertugas melakukan verifikasi pesanan dan pembayaran melalui fitur *Manage Orders & Check Payment*, serta mengawasi pergerakan armada pengiriman di lapangan secara *real-time* melalui fitur *Monitor Courier Location*.

Selanjutnya, aktor Pelanggan (*Customer*) berinteraksi melalui aplikasi *mobile* untuk kebutuhan belanja. Aktivitas utamanya mencakup menelusuri katalog produk dan melakukan pemesanan (*Place Order*) yang secara sistem mencakup proses pembayaran. Untuk meningkatkan pengalaman pengguna, Pelanggan disediakan fitur interaktif yaitu *Live Driver Location* yang memungkinkan pelacakan posisi kurir di peta, serta fitur *In-App Chat* yang berfungsi sebagai sarana komunikasi dua arah dengan kurir mengenai detail pengiriman barang.

Terakhir, aktor Kurir menggunakan aplikasi khusus untuk menunjang operasional logistik harian. Kurir dapat melihat daftar tugas pengiriman dan turut terlibat dalam fitur *In-App Chat* untuk berkoordinasi

dengan pelanggan. Saat menjalankan tugas, kurir berinteraksi dengan fitur *Live Driver Location* untuk menyiarkan posisi terkini mereka agar dapat dipantau oleh sistem. Rangkaian proses pengiriman diakhiri dengan pemindaian *QR Code* dan pengunggahan bukti penerimaan barang (*Upload Proof of Delivery*) sebagai validasi akhir tugas.

### C. Activity Diagram

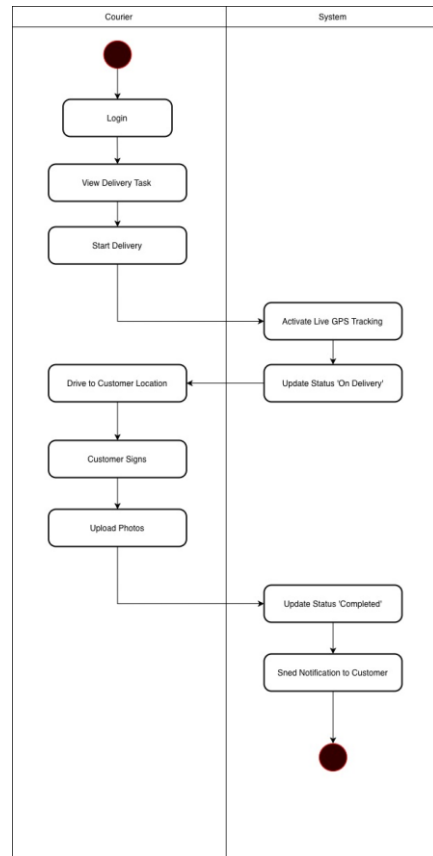


**Gambar 3**  
**Activity Diagram Ordering Process**

Berdasarkan Gambar 3, alur aktivitas dimulai pada kolom *Customer* saat pengguna membuka aplikasi dan melakukan *Login*. Pengguna kemudian menelusuri katalog produk (*Browse Product Catalog*) dan memilih produk yang diinginkan untuk dimasukkan ke dalam keranjang (*Add Product to Cart*). Setelah pengguna melanjutkan ke tahap *Checkout*, sistem akan memeriksa ketersediaan stok barang (*Check Stock*). Jika stok tersedia, sistem menghitung total biaya transaksi (*Calculate Total Payment*) yang mencakup harga barang dan biaya pengiriman *flat rate*.

Selanjutnya, pengguna melakukan proses pembayaran (*Payment Process*) melalui gerbang pembayaran yang tersedia. Sistem kemudian memverifikasi status transaksi tersebut (*Verify Payment Status*). Apabila pembayaran dinyatakan sukses, pengguna diminta untuk memilih jadwal pengiriman (*Select Delivery Schedule*) sesuai slot yang tersedia. Setelah jadwal ditentukan, sistem akan memfinalisasi pesanan dengan

memperbarui data stok (*Update Stock*) dan menyimpan data transaksi sebagai tanda bahwa proses pemesanan telah selesai.

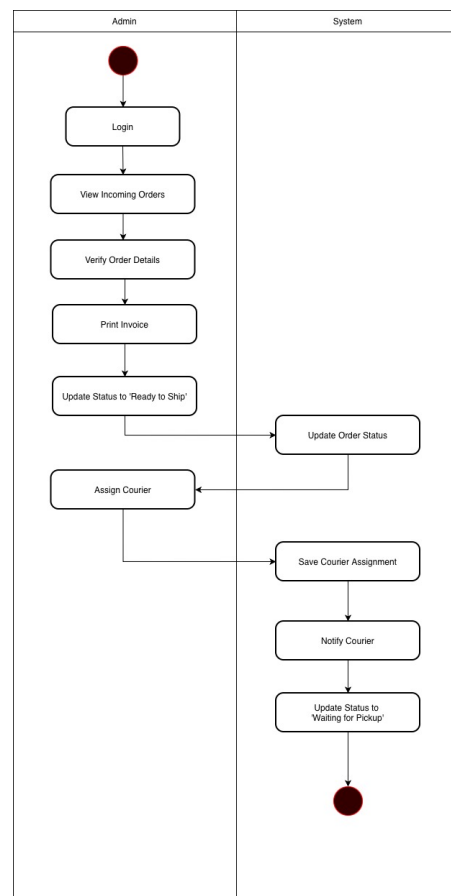


**Gambar 4**  
**Activity Diagram Delivery Process**

Selanjutnya, Gambar 4 menggambarkan alur aktivitas pada sisi operasional logistik. Aktivitas dimulai pada *Courier* ketika kurir masuk ke dalam aplikasi (*Login*) dan melihat daftar tugas pengiriman harian (*View Delivery Task*). Setelah kurir memilih tugas dan menekan tombol mulai (*Start Delivery*), sistem secara otomatis mengaktifkan pelacakan lokasi (*Activate Live GPS Tracking*) dan memperbarui status pesanan menjadi "On Delivery". Hal ini memungkinkan posisi kurir terpantau secara *real-time*.

Kurir kemudian menuju lokasi pelanggan (*Drive to Customer Location*). Setibanya di tujuan, dilakukan proses serah terima barang yang divalidasi dengan tanda tangan digital pelanggan (*Customer Signs*)

menggunakan teknologi *Canvas API* pada perangkat kurir. Sebagai bukti akhir, kurir mengunggah foto penerimaan barang (*Upload Photos*). Setelah data bukti terunggah, sistem secara otomatis memfinalisasi pesanan dengan memperbarui status menjadi "Completed" dan mengirimkan notifikasi kepada pelanggan (*Send Notification to Customer*) bahwa paket telah diterima dengan baik.



**Gambar 5**

#### Activity Diagram of Order Fulfillment

Terakhir, Gambar 5 menggambarkan aktivitas manajemen pesanan (*Order Fulfillment*) yang dilakukan oleh Admin sebelum barang diserahkan kepada kurir. Aktivitas dimulai pada kolom *Admin* saat admin masuk ke *dashboard* (*Login*) dan memeriksa daftar pesanan baru yang masuk (*View Incoming Orders*). Admin kemudian memverifikasi rincian pesanan (*Verify Order Details*) untuk memastikan kesesuaian barang, lalu



mencetak faktur atau label pengiriman (*Print Invoice*) sebagai dokumen fisik paket.

Setelah paket siap secara fisik, Admin memperbarui status pesanan menjadi "Ready to Ship". Sistem akan mencatat perubahan status tersebut (*Update Order Status*). Langkah krusial selanjutnya adalah penetapan armada, di mana Admin memilih dan menugaskan kurir spesifik untuk pesanan tersebut (*Assign Courier*). Sistem kemudian menyimpan data penugasan (*Save Courier Assignment*), memperbarui status menjadi "Waiting for Pickup", dan secara otomatis mengirimkan notifikasi tugas baru kepada kurir (*Notify Courier*), yang menjadi pemicu awal bagi aktivitas pengiriman kurir.

#### D. Fitur Program

Berdasarkan analisis kebutuhan dan arsitektur yang telah dirancang, pengembangan sistem akan dibagi menjadi tiga modul utama yang saling terintegrasi. Berikut adalah rincian spesifikasi fungsional yang akan dibangun pada tahap implementasi:

##### 1. Modul Web Administrator

Modul ini dikembangkan menggunakan *SvelteKit* dan ditujukan bagi pengelola toko untuk memantau operasional bisnis dan logistik secara terpusat. Fitur-fitur utama meliputi:

- **Dashboard Monitoring**  
Menyajikan visualisasi data ringkas mengenai total pendapatan, jumlah pesanan aktif, dan peringatan dini stok barang yang telah mencapai batas *Reorder Point* (ROP).
- **Manajemen Inventori**  
Memungkinkan admin untuk mengelola data produk (tambah, ubah, hapus), memantau jumlah stok fisik, dan mengatur harga jual.
- **Manajemen Pemenuhan Pesanan**  
Auto-dispatch Engine: Fitur utama yang secara otomatis membagi tugas pengiriman kepada kurir yang sedang aktif (Online) dan memiliki beban kerja paling rendah, sehingga Admin tidak perlu memilih kurir secara manual satu per satu.
- **Validasi Pengiriman**  
Admin bertindak sebagai validator akhir dengan memeriksa bukti foto dan tanda tangan digital yang dikirimkan kurir sebelum mengubah status pesanan menjadi selesai (*Completed*).
- **Manajemen Multi-satuan**  
Fitur ini memungkinkan admin untuk mengelola stok barang dengan variasi satuan kemasan, khususnya untuk komoditas seperti beras dan gula. Admin dapat menetapkan nilai konversi yang spesifik (contoh: **1 Sak = 50 Kg**), sehingga setiap transaksi penjualan dalam satuan Sak akan secara otomatis memotong stok pada satuan dasar (Kilogram) secara akurat di dalam database.

Commented [KS4]: Hasil Sidang Nomor 5

## 2. Modul Mobile Customer

Modul ini dikembangkan menggunakan *Flutter* untuk memberikan pengalaman belanja yang responsif bagi pengguna akhir. Fitur-fitur utama meliputi:

- **Katalog & Keranjang Belanja**  
Antarmuka untuk menelusuri daftar produk, melihat detail spesifikasi, dan mengelola keranjang belanja sebelum transaksi.
- **Integrasi Pembayaran Digital**  
Sistem terhubung dengan *Payment Gateway* (Midtrans) yang memungkinkan pelanggan membayar melalui berbagai metode (Transfer Bank, E-Wallet) dengan kalkulasi biaya pengiriman *flat rate* secara otomatis.
- **Pemilihan Jadwal Pengiriman**  
Setelah pembayaran sukses, pelanggan diberikan akses untuk memilih slot waktu pengiriman yang tersedia sesuai preferensi mereka.
- **Pelacakan Pesanan**  
Pelanggan dapat memantau posisi kurir secara *real-time* melalui peta digital interaktif berbasis OpenStreetMap saat pesanan sedang dalam perjalanan, tanpa membebani kuota API berbayar.
- **Riwayat Transaksi & Status Pesanan:**  
Fitur ini menampilkan daftar lengkap riwayat belanja pengguna, mulai dari pesanan yang sedang diproses, dikirim, hingga yang telah selesai atau dibatalkan. Pengguna dapat melihat detail nota digital, rincian produk yang dibeli, serta status pengiriman terakhir untuk keperluan *repurchase* (pembelian ulang) atau komplain.

## 3. Modul Mobile Kurir

Modul ini dikembangkan menggunakan *Flutter* dengan fokus pada operasional lapangan dan validasi data lokasi. Fitur-fitur utama meliputi:

- **Manajemen Tugas Pengiriman**  
Kurir dapat melihat daftar tugas harian yang telah ditugaskan oleh sistem, lengkap dengan detail alamat dan kontak penerima.
- **Navigasi & Live Location**  
Aplikasi menggunakan koneksi WebSocket yang persisten untuk mengirimkan lokasi terkini kurir ke server secara terus-menerus. Selain itu, fitur ini juga berfungsi menerima notifikasi penugasan pesanan baru secara instan (*In-App Alert*) tanpa perlu melakukan *refresh* halaman manual.
- **Bukti Pengiriman Digital**  
Fitur validasi penerimaan barang yang memanfaatkan *Canvas API*. Kurir diwajibkan mengambil foto lokasi dan meminta tanda tangan digital pelanggan langsung di layar *smartphone* sebagai bukti sah serah terima barang.

#### E. Batasan Sistem

Untuk memastikan pengembangan sistem tetap terarah dan sesuai dengan rancangan arsitektur yang telah ditetapkan, penulis menetapkan batasan-batasan sebagai berikut:

1. Ruang Lingkup Operasional: Sistem difokuskan pada layanan pengiriman dalam kota (*intracity*) yang dikelola oleh armada internal toko. Sistem ini berdiri sendiri dan tidak terintegrasi dengan API ekspedisi pihak ketiga (seperti JNE atau J&T).
2. Integrasi Pembayaran: Sistem menerapkan metode pembayaran non-tunai (*Cashless*) secara penuh melalui integrasi Payment Gateway Midtrans. Sistem tidak melayani pembayaran tunai di tempat (*Cash on Delivery*), dan seluruh status transaksi diverifikasi otomatis oleh sistem.
3. Sistem memanfaatkan peta berbasis OpenStreetMap (OSM) dan layanan OpenRouteService untuk fitur navigasi dan pelacakan, menggantikan penggunaan layanan berbayar. Fitur *Live Tracking* tersedia untuk memantau posisi kurir secara *real-time* saat status pengiriman aktif, serta validasi lokasi akhir menggunakan koordinat GPS saat bukti foto diunggah.
4. Manajemen Penugasan: Sistem menangani penugasan kurir secara otomatis (*Auto-dispatch*) berdasarkan lokasi dan ketersediaan armada. Penentuan urutan rute pengiriman dilakukan menggunakan algoritma Nearest Neighbor yang bersifat sekuensial (berurutan dari titik terdekat). Pengelompokan daerah pengiriman dilakukan secara otomatis berdasarkan radius jarak dari toko.
5. Protokol Komunikasi & Notifikasi:  
Sistem menggunakan protokol WebSocket untuk fitur *real-time* (pelacakan lokasi dan notifikasi). Karena standar operasional kurir mewajibkan aplikasi selalu terbuka (*standby*) selama jam kerja, maka notifikasi yang diimplementasikan hanya bersifat In-App Notification (berjalan saat aplikasi aktif/foreground) dan tidak mencakup Push Notification saat aplikasi ditutup total.
6. Manajemen Multi-satuan  
Fitur ini memungkinkan admin untuk mengelola stok barang dengan variasi satuan kemasan, khususnya untuk komoditas seperti beras dan gula. Admin dapat menetapkan nilai konversi yang spesifik (contoh: 1 Sak = 50 Kg), sehingga setiap transaksi penjualan dalam satuan Sak akan secara otomatis memotong stok pada satuan dasar (Kilogram) secara akurat di dalam database.
7. Skala Pengujian Sistem:  
Pengujian fungsionalitas dan User Acceptance Test (UAT) akan dilakukan dengan skala simulasi operasional terbatas. Target responden pengujian ditetapkan minimal 25 pengguna sebagai Customer dan 3 pengguna sebagai Kurir untuk memvalidasi alur pemesanan hingga penyelesaian pengiriman barang.

Commented [KS5]: Hasil Sidang Nomor 6

Platform Pengembangan:

- Aplikasi sisi Pelanggan dan Kurir dibangun menggunakan *framework* Flutter dan dioptimalkan untuk perangkat berbasis Android.
- Aplikasi sisi Admin dibangun menggunakan *framework* SvelteKit dan diakses melalui peramban web *desktop*.

#### F. Aplikasi Perbandingan

Pada subbab ini, dilakukan perbandingan fitur antara sistem yang dikembangkan dengan aplikasi Lalamove, yang merupakan pemimpin pasar dalam layanan pengiriman on-demand. Perbandingan ini bertujuan untuk menonjolkan nilai tambah dari sistem internal yang dibangun, khususnya dalam aspek integrasi data inventori.

**Tabel 1**  
**Tabel Perbandingan Fitur**

| Fitur                           | Lalamove | Sistem Saya |
|---------------------------------|----------|-------------|
| Pelacakan Lokasi (Real-time)    | ✓        | ✓           |
| Optimasi Rute Multi-stop        | ✓        | ✓           |
| Bukti pengiriman (foto & Ttd)   | ✓        | ✓           |
| Jangkauan Area Layanan Nasional | ✓        | ✗           |
| Ketersediaan Driver 24 jam      | ✓        | ✗           |
| Integrasi Otomatis dengan stok  | ✗        | ✓           |

Berdasarkan Tabel 1, sistem yang dibangun memiliki standar fitur operasional yang setara dengan Lalamove, meliputi kemampuan pelacakan lokasi real-time, optimasi rute multi-stop, serta validasi bukti pengiriman digital (foto & tanda tangan). Hal ini menunjukkan bahwa secara teknis, sistem mampu menangani kebutuhan pengiriman barang dengan standar industri logistik modern.

Namun, tabel juga memperlihatkan batasan operasional sistem usulan (bertanda X) pada aspek jangkauan area nasional dan ketersediaan pengemudi 24 jam. Hal ini wajar karena sistem ini dirancang khusus untuk manajemen armada internal toko yang memiliki jam kerja tetap, bukan untuk layanan crowdsourcing umum seperti Lalamove.

Keunggulan kompetitif utama sistem usulan terletak pada fitur Integrasi Otomatis dengan Stok (bertanda ✓ pada Sistem Saya, namun X pada Lalamove). Lalamove beroperasi sebagai entitas terpisah yang tidak memiliki akses ke data gudang pengirim. Sebaliknya, sistem ini menghubungkan proses pengiriman langsung dengan database inventori, sehingga stok barang akan terpotong secara otomatis dan real-time begitu status pengiriman terkonfirmasi selesai.

Commented [KS6]: Hasil Sidang Nomor 4

## **V. Rencana Uji Coba**

Untuk memastikan bahwa sistem manajemen toko dan logistik internal yang dikembangkan berjalan sesuai dengan standar operasional yang diharapkan, perlu dilakukan serangkaian uji coba secara menyeluruh. Oleh sebab itu, dua metode pengujian perangkat lunak akan diterapkan untuk menjamin keberhasilan implementasi sistem ini, yaitu Functionality Testing dan User Acceptance Testing (UAT).

Functionality Testing dilakukan untuk memverifikasi bahwa seluruh modul dan fitur dalam sistem, baik pada sisi antarmuka web Admin maupun aplikasi mobile Kurir dan Pelanggan, berfungsi sesuai dengan spesifikasi teknis yang dirancang. Pengujian ini mencakup validasi alur proses bisnis utama, mulai dari manajemen data produk dan stok oleh admin, proses transaksi dan pembayaran digital oleh pelanggan, fitur pelacakan lokasi secara real-time, hingga mekanisme validasi pengiriman menggunakan tanda tangan digital. Pengujian dilakukan menggunakan skenario Black Box Testing untuk mengidentifikasi potensi bug, kesalahan kalkulasi stok, atau kegagalan integrasi API yang mungkin terjadi sebelum sistem diserahkan kepada pengguna akhir.

User Acceptance Testing (UAT) dilakukan untuk mengevaluasi tingkat penerimaan dan kepuasan pengguna terhadap solusi yang ditawarkan sistem dalam lingkungan operasional yang sebenarnya. Uji coba ini melibatkan 1 orang Admin Toko, 3 orang Kurir armada internal, serta 25 orang Pelanggan sukarelawan untuk mensimulasikan transaksi nyata. UAT dilaksanakan dalam kurun waktu 1 bulan dengan skenario penggunaan mencakup siklus lengkap: pembelian barang, penugasan kurir oleh admin, proses pengantaran, hingga validasi bukti serah terima di lokasi tujuan. Selama periode pengujian, seluruh partisipan diminta untuk memberikan penilaian dan umpan balik mengenai kemudahan antarmuka dan kinerja sistem melalui kuesioner skala Likert yang disebarakan menggunakan Google Form. Data yang diperoleh kemudian akan dianalisis untuk mengukur efektivitas sistem dalam menangani manajemen logistik toko secara mandiri.

## **VI. Daftar Pustaka**

- [1] A. S. Tanenbaum and M. van Steen, Distributed Systems: Principles and Paradigms, 3rd ed. CreateSpace Independent Publishing Platform, 2017.
- [2] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York: McGraw-Hill Education, 2019.
- [3] J. Heizer, B. Render, and C. Munson, Operations Management: Sustainability and Supply Chain Management, 12th ed. Boston: Pearson, 2017.
- [4] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. Info. & Comp. Sci., Univ. of California, Irvine, CA, 2000.

- [5] K. C. Laudon and C. G. Traver, E-Commerce 2021-2022: Business, Technology, Society, 16th ed. Pearson, 2021.
- [6] J. Schiller and A. Voisard, Location-Based Services. Morgan Kaufmann, 2004.
- [7] NestJS, "NestJS - A progressive Node.js framework," NestJS Documentation. [Online]. Available: <https://docs.nestjs.com/>. [Accessed: Jan. 25, 2025].
- [8] Svelte, "SvelteKit • Web development, streamlined," SvelteKit Documentation. [Online]. Available: <https://kit.svelte.dev/>. [Accessed: Jan. 25, 2025].
- [9] Google Developers, "Flutter - Build apps for any screen," Flutter Documentation. [Online]. Available: <https://flutter.dev/>. [Accessed: Jan. 25, 2025].
- [10] Prisma, "Prisma | Next-generation ORM for Node.js & TypeScript," Prisma Documentation. [Online]. Available: <https://www.prisma.io/>. [Accessed: Jan. 25, 2025].
- [11] PostgreSQL Global Development Group, "PostgreSQL: The World's Most Advanced Open Source Relational Database," PostgreSQL Documentation. [Online]. Available: <https://www.postgresql.org/>. [Accessed: Jan. 25, 2025].
- [12] OpenStreetMap Foundation, "OpenStreetMap," [Online]. Available: <https://www.openstreetmap.org>. [Accessed: Feb. 10, 2026].
- [13] Heidelberg Institute for Geoinformation Technology, "OpenRouteService API Documentation," [Online]. Available: <https://openrouteservice.org>. [Accessed: Feb. 10, 2026].
- [14] Midtrans, "Technical Documentation Midtrans," Midtrans Docs. [Online]. Available: <https://docs.midtrans.com/>. [Accessed: Jan. 25, 2025]. [14] MDN Web Docs, "Canvas API," Mozilla Developer Network. [Online]. Available: [https://developer.mozilla.org/en-US/docs/Web/API/Canvas\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API). [Accessed: Jan. 25, 2025].